GenAI analysis - Assignment 4

Gen-AI # 1 - Google Gemini

1. **I accessed this on my computer**
2. **My prompt:**

I am working on a college EECS 348 Software Engineering I programming assignment in C++. I need to create a recursive, object-oriented Sudoku solver using brute-force search with backtracking.
Assignment Requirements:
Sudoku uses a 9×9 grid divided into nine 3×3 blocks.
Every row, column, and 3×3 block must contain the digits 1–9 exactly once.
I am given five input files: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
Each file contains a partially completed Sudoku puzzle. Existing digits are provided, and underscores (_) represent empty cells.
The program must use recursive depth-first search with backtracking to find solutions.
The program must use an object-oriented design.
If no solution exists, the program must print No solution found.
If multiple valid solutions exist, the program must print all of them, not just the first one.
For each puzzle, the output must include the puzzle filename, the original puzzle printed out, and its solution or all solutions.
I am also given solution1.txt, which contains a solution to puzzle1.txt and can be used to test my program.

Your Tasks:
Write a complete program that satisfies all the assignment requirements.
If the required programming language is not specified, ask me which language my instructor requires before writing the code.
Use simple, beginner-friendly code appropriate for a college programming course. Avoid unnecessarily advanced techniques.
Explain the object-oriented design, including the class and its methods.
Explain how the recursive backtracking algorithm works and how the program determines which numbers are valid for each empty cell.
Make sure the program correctly restores empty cells when a candidate number fails, so it can try other possibilities.
Make sure the program continues searching after finding a solution so it can print every valid solution.
Do not overwrite the original puzzle's given numbers.
Include appropriate input validation and handle invalid or unsolvable puzzles.
Explain how to compile and run the program with the five input files.
Provide a testing plan using solution1.txt to verify the solution for puzzle1.txt, along with tests for the other puzzles.
Include example output formatting, but do not invent actual solutions for puzzle files you have not seen.

Review the finished program against each assignment requirement and identify any potential errors or limitations. Assume I am a beginner-level college programmer. Keep the implementation straightforward and explain the important parts of the code so I can understand how it works.

3. **The given code:**

```
#include <iostream>
#include <vector>
#include <string>
#include <fstream>

class SudokuSolver {
private:
    char board[9][9];

    // Helper method to check if placing num at (row, col) is valid
    bool isValid(int row, int col, char num) const {
        // Check row and column
        for (int i = 0; i < 9; ++i) {
            if (board[row][i] == num) return false;
            if (board[i][col] == num) return false;
        }

        // Check 3x3 subgrid
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (board[startRow + r][startCol + c] == num) return false;
            }
        }

        return true;
    }

    // Recursive backtracking method designed to find ALL solutions
    bool solveRecursive(int row, int col, int& solutionCount) {
        // Base case: If row has reached 9, we have successfully filled the board
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution #" << solutionCount << ":\n";
            printBoard();
            std::cout << "\n";
```

```
            return false; // Return false to force backtracking and find all solutions
        }

        // Determine coordinates for the next cell
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If the current cell is already filled (part of the original puzzle), skip it
        if (board[row][col] != '_') {
            return solveRecursive(nextRow, nextCol, solutionCount);
        }

        bool foundAny = false;
        // Try digits '1' through '9'
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                board[row][col] = num; // Make choice

                // Recurse to next cell
                if (solveRecursive(nextRow, nextCol, solutionCount)) {
                    foundAny = true;
                }

                board[row][col] = '_'; // Backtrack: undo choice and restore empty cell
            }
        }

        return foundAny;
    }

public:
    // Constructor initializes board to empty
    SudokuSolver() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                board[i][j] = '_';
            }
        }
    }

    // Load puzzle from file with input validation
    bool loadPuzzle(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
```

```
        std::cerr << "Error: Could not open file " << filename << "\n";
        return false;
    }

    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (!(file >> board[i][j])) {
                std::cerr << "Error: File format invalid or incomplete in " << filename << "\n";
                return false;
            }
            // Basic validation: check if character is valid (digit 1-9 or underscore)
            if ((board[i][j] < '1' || board[i][j] > '9') && board[i][j] != '_') {
                std::cerr << "Error: Invalid character '" << board[i][j]
                          << "' found in " << filename << ". Expected '1'-'9' or '_'.\n";
                return false;
            }
        }
    }
    return true;
}

// Print the Sudoku board with clear grid formatting
void printBoard() const {
    for (int i = 0; i < 9; ++i) {
        if (i > 0 && i % 3 == 0) {
            std::cout << "------+-------+------\n";
        }
        for (int j = 0; j < 9; ++j) {
            if (j > 0 && j % 3 == 0) {
                std::cout << "| ";
            }
            std::cout << board[i][j] << " ";
        }
        std::cout << "\n";
    }
}

// Public solve driver method
void solve(const std::string& filename) {
    std::cout << "========================================\n";
    std::cout << "Processing Puzzle: " << filename << "\n";
    std::cout << "========================================\n";

    if (!loadPuzzle(filename)) {
```

```
            std::cout << "Skipping " << filename << " due to load errors.\n\n";
            return;
        }

        std::cout << "Original Puzzle:\n";
        printBoard();
        std::cout << "\n";

        int solutionCount = 0;
        solveRecursive(0, 0, solutionCount);

        if (solutionCount == 0) {
            std::cout << "No solution found.\n\n";
        } else {
            std::cout << "Total solutions found for " << filename << ": " << solutionCount << "\n\n";
        }
    }
};

int main() {
    SudokuSolver solver;

    // Array containing the five required input files
    std::string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Solve each puzzle sequentially
    for (int i = 0; i < 5; ++i) {
        solver.solve(puzzleFiles[i]);
    }

    return 0;
}
```

Gen-AI # 2 Copilot

1. **Accessed Copilot on my computer**
2. **Prompt:**

I am working on a college EECS 348 Software Engineering I programming assignment in C++. I need to create a recursive, object-oriented Sudoku solver using brute-force search with backtracking.
Assignment Requirements:
Sudoku uses a 9×9 grid divided into nine 3×3 blocks.
Every row, column, and 3×3 block must contain the digits 1–9 exactly once.
I am given five input files: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
Each file contains a partially completed Sudoku puzzle. Existing digits are provided, and underscores (_) represent empty cells.
The program must use recursive depth-first search with backtracking to find solutions.
The program must use an object-oriented design.
If no solution exists, the program must print No solution found.
If multiple valid solutions exist, the program must print all of them, not just the first one.
For each puzzle, the output must include the puzzle filename, the original puzzle printed out, and its solution or all solutions.
I am also given solution1.txt, which contains a solution to puzzle1.txt and can be used to test my program.

Your Tasks:
Write a complete program that satisfies all the assignment requirements.
If the required programming language is not specified, ask me which language my instructor requires before writing the code.
Use simple, beginner-friendly code appropriate for a college programming course. Avoid unnecessarily advanced techniques.
Explain the object-oriented design, including the class and its methods.
Explain how the recursive backtracking algorithm works and how the program determines which numbers are valid for each empty cell.
Make sure the program correctly restores empty cells when a candidate number fails, so it can try other possibilities.
Make sure the program continues searching after finding a solution so it can print every valid solution.
Do not overwrite the original puzzle's given numbers.
Include appropriate input validation and handle invalid or unsolvable puzzles.
Explain how to compile and run the program with the five input files.
Provide a testing plan using solution1.txt to verify the solution for puzzle1.txt, along with tests for the other puzzles.
Include example output formatting, but do not invent actual solutions for puzzle files you have not seen.
Review the finished program against each assignment requirement and identify any potential errors or limitations. Assume I am a beginner-level college programmer. Keep the implementation straightforward and explain the important parts of the code so I can understand how it works.

3. Given Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
using namespace std;
class Sudoku
{
private:
		int board[9][9];
		bool fixed[9][9];	// True for original given numbers
		int solutionCount;
public:
		Sudoku()
		{
		solutionCount = 0;
		for (int r = 0; r < 9; r++)
		{
		for (int c = 0; c < 9; c++)
		{
		board[r][c] = 0;
		fixed[r][c] = false;
		}
	}
}
		bool loadPuzzle(string filename)
		{
		ifstream file(filename);
		if (!file)
		{
		cout << "Error: Cannot open " << filename << endl;
		return false;
		}
		char ch;
		for (int r = 0; r < 9; r++)
		{
		for (int c = 0; c < 9; c++)
		{
		if (!(file >> ch))
		{
		cout << "Invalid puzzle format." << endl;
		return false;
		}
		if (ch == '_')
		{
		board[r][c] = 0;
		fixed[r][c] = false;
		}
		else if (ch >= '1' && ch <= '9')
		{
		board[r][c] = ch - '0';
		fixed[r][c] = true;
		}
		else
		{
		cout << "Invalid character in puzzle." << endl;
		return false;
		}
		}
		}
		return true;
		}
```

```
void printBoard()
{
for (int r = 0; r < 9; r++)
{
for (int c = 0; c < 9; c++)
{
if (board[r][c] == 0)
cout << "_ ";
else
cout << board[r][c] << " ";
}
cout << endl;
}
}
bool isValid(int row, int col, int num)
{
// Check row
for (int c = 0; c < 9; c++)
{
if (board[row][c] == num)
return false;
}
// Check column
for (int r = 0; r < 9; r++)
{
if (board[r][col] == num)
return false;
}
// Check 3x3 block
int startRow = (row / 3) * 3;
int startCol = (col / 3) * 3;
for (int r = startRow; r < startRow + 3; r++)
{
for (int c = startCol; c < startCol + 3; c++)
{
if (board[r][c] == num)
return false;
}
}
return true;
}
bool initialPuzzleValid()
{
for (int row = 0; row < 9; row++)
{
for (int col = 0; col < 9; col++)
{
if (board[row][col] != 0)
{
int value = board[row][col];
board[row][col] = 0;
bool valid = isValid(row, col, value);
board[row][col] = value;
if (!valid)
        return false;
}
}
}
return true;
}
void solve()
```

```
        {
        solveRecursive();
        if (solutionCount == 0)
        {
        cout << "No solution found." << endl;
        }
        }
        void solveRecursive()
        {
        int row = -1;
        int col = -1;
        bool emptyFound = false;
        // Find first empty cell
        for (int r = 0; r < 9 && !emptyFound; r++)
        {
        for (int c = 0; c < 9; c++)
        {
        if (board[r][c] == 0)
        {
        row = r;
        col = c;
        emptyFound = true;
        break;
        }
        }
        }
        // No empty cells means solution found
        if (!emptyFound)
        {
        solutionCount++;
        cout << "\nSolution " << solutionCount << ":" << endl;
        printBoard();
        return; // continue searching for more solutions
        }
        // Try numbers 1-9
        for (int num = 1; num <= 9; num++)
        {
        if (isValid(row, col, num))
        {
        board[row][col] = num;
        solveRecursive();
        // Backtrack
        board[row][col] = 0;
        }
        }
        }
        int getSolutionCount()
        {
        return solutionCount;
        }
};
int main()
{
        vector<string> files =
        {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
        };
```

```
        for (string filename : files)
        {
        cout << "\n====================================" << endl;
        cout << "Puzzle File: " << filename << endl;
        cout << "====================================" << endl;
        Sudoku puzzle;
        if (!puzzle.loadPuzzle(filename))
        {
        continue;
        }
        cout << "\nOriginal Puzzle:\n" << endl;
        puzzle.printBoard();
        if (!puzzle.initialPuzzleValid())
        {
        cout << "\nInvalid puzzle." << endl;
        continue;
        }
        cout << "\nSearching for solutions...\n" << endl;
        puzzle.solve();
        cout << "\nTotal Solutions Found: "
        << puzzle.getSolutionCount()
        << endl;
        }
        return 0;
}
```

Code Comparison:

1. Correctness

Gemini uses recursive backtracking and correctly checks rows, columns, and 3×3 blocks. It also restores empty cells and prints all solutions. It does not check whether the original puzzle contains duplicate numbers, and its Boolean return value is unnecessary. Copilot also uses backtracking and finds all solutions. Its initialPuzzleValid() method checks for duplicate numbers in the original puzzle making it more reliable but it does not check for extra characters after the expected 81.

2. Execution Time

Gemini passes the row and column directly between recursive calls avoiding repeated searches for the next empty cell. It does not prioritize cells with fewer possible values which might reduce unnecessary attempts. Copilot searches the board for the next empty cell during every recursive call. Both programs have exponential worst-case running times.

3. Space Complexity

Gemini uses a 9×9 character array and a recursive call stack. Its space complexity is O(1) for a fixed size Sudoku board. It prints solutions immediately. Copilot uses a 9×9 integer array, an additional Boolean array, and a recursive call stack. Its space complexity is also O(1) for a fixed size board but it uses slightly more memory.

4. Maintainability

Gemini separates its code into clear methods for loading, checking, solving, and printing. The unnecessary Boolean return value makes the recursive method slightly more confusing though. Copilot has clearly separated methods and includes initial puzzle validation but the unused fixed array adds unnecessary complexity.

Selection:

I chose Gemini's code because it uses a straightforward recursive backtracking algorithm and passes the row and column directly between recursive calls. This avoids repeatedly searching for the next empty cell and makes the program easier to follow. It also uses less memory than Copilot's code because it does not need an extra fixed array.

Code Improvement:

1. Correctness

I would add a method to check whether the original puzzle contains duplicate numbers in any row, column, or block. I would also improve input validation.

2. Execution

I would keep Gemini's approach of passing row and column coordinates directly between recursive calls. I could also improve the search by choosing the empty cell with the fewest valid candidates first.

3. Space Complexity

I would keep the single 9×9 character array and avoid storing every solution in memory. The program's space complexity would remain O(1)

4. Maintainability

I would simplify the recursive method by removing its unnecessary Boolean return value and use a clearer method structure. I would also do comments and error messages to make the program easier to understand.